INFORME DE INVESTIGACIÓN A PROFUNDIDAD (DEEP RESEARCH)

Arquitectura de Autenticación con JSON Web Tokens (JWT) y Almacenamiento Seguro en Clientes Web

Proyecto: DarkKitchen Hub - Sistema de Gestión de Cocinas Ocultas y Despacho Multiplataforma
Materia: Mini Proyecto 1 - Día 2: Implementación de Autenticación y Rutas Protegidas
Autor: Luis Alberto Méndez Pozo
Fecha: Noviembre 2026

1. Resumen Ejecutivo

El presente documento constituye el informe de investigación técnica previa a la incorporación de la capa de autenticación y autorización en el sistema empresarial DarkKitchen Hub. Con la transición hacia una arquitectura protegida en el Día 2, la API REST en Node.js/Express restringe el acceso a sus recursos exclusivamente a clientes autenticados. Este informe analiza los fundamentos criptográficos de los JSON Web Tokens (JWT), su generación y validación en el servidor, los mecanismos de expiración y renovación, así como un análisis comparativo de vectores de ataque (XSS vs. CSRF) según el almacenamiento en React, concluyendo con la justificación técnica de la decisión adoptada.

2. Fundamentos de JSON Web Token (JWT)

2.1 Definición y Propósito

Un JSON Web Token (estandarizado bajo el RFC 7519) es un estándar abierto, compacto y autónomo (stateless) para transmitir información de forma segura entre partes como un objeto JSON. Al ser autónomo, el token contiene en sí mismo toda la información necesaria sobre la identidad del usuario, eliminando la necesidad de que el servidor mantenga sesiones en memoria o consulte la base de datos en cada petición para comprobar si una sesión sigue abierta.

2.2 Estructura Anatómica del Token

Un JWT está formado por tres segmentos independientes codificados en Base64Url y concatenados mediante puntos (`.`):

- **Header (Encabezado):** Indica el tipo de token (`JWT`) y el algoritmo criptográfico utilizado para la firma digital (por ejemplo, HMAC SHA-256 o RSA).

```
None
{
  "alg": "HS256",
  "typ": "JWT"
}
```

- **Payload (Carga Útil o Reclamaciones):** Contiene las declaraciones (claims) sobre la entidad (usuario) y metadatos adicionales. Incluye claims registrados estándar (como `exp` para expiración, `iat` para emisión) y claims personalizados (como el `id` y `correo` del usuario).

n
{
"id": 15,
"correo": "admin@darkkitchen.com",
"iat": 1730800000,
"exp": 1730828800
}

```
None

*Nota crítica de seguridad:* El payload no está cifrado ni oculto; únicamente está codificado en Base64Url. Cualquier intermediario o cliente puede decodificarlo e inspeccionarlo. Por este motivo, nunca deben incluirse contraseñas, hashes, datos de tarjetas bancarias o información altamente sensible en esta sección.

* **Signature (Firma Digital):** Se genera tomando el Header codificado, el Payload codificado, una clave secreta privada (`JWT_SECRET`) conocida únicamente por el servidor backend, y aplicando el algoritmo especificado en el Header:

```text
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  JWT_SECRET
)
```
```

La firma garantiza la integridad del mensaje: si un cliente altera fraudulentamente el `id` o el rol en el payload, la firma resultante ya no coincidirá con la calculada por el servidor, invalidando la solicitud al instante.

3. Implementación en Node.js y Express

3.1 Emisión del Token en Inicio de Sesión (`POST /api/auth/login`)

En el entorno Node.js, la librería estándar de la industria es `jsonwebtoken`. Al validar las credenciales del usuario con `bcrypt`, el backend firma el token utilizando la función `jwt.sign()`:

```javascript
const jwt = require('jsonwebtoken');

const token = jwt.sign(
  { id: usuario.id, correo: usuario.correo },
  process.env.JWT_SECRET,
  { expiresIn: '8h' }
);
```

3.2 Protección de Rutas mediante Middleware de Autenticación

Para restringir las rutas del CRUD, se implementa un middleware interceptor que inspecciona los encabezados HTTP entrantes:

- Extrae el encabezado `Authorization`.
- Valida que el formato corresponda al esquema `Bearer <token>`.
- Si el encabezado no existe o no tiene el prefijo `Bearer`, responde inmediatamente con código HTTP `401 Unauthorized` y el cuerpo `{"error": "No autorizado"}`.
- Ejecuta `jwt.verify(token, process.env.JWT_SECRET)`. Si el token es legítimo y no ha caducado, inyecta la información decodificada en `req.usuario` y cede el control al siguiente controlador mediante `next()`.
- Si el token está adulterado o expirado, captura el error y retorna código HTTP `401`.

4. Ciclo de Vida, Expiración y Renovación de Tokens

4.1 Política de Expiración

Un token sin fecha de vencimiento representa un fallo severo de seguridad: si es interceptado o filtrado, el atacante mantendría acceso indefinido. Por convención, los tokens de acceso (Access Tokens) deben contar con un tiempo de vida acotado (típicamente entre 15 minutos y 8 horas, dependiendo de la sensibilidad del sistema). En sistemas de operación interna continua, como DarkKitchen Hub, una expiración de 4 a 8 horas cubre la jornada del turno de cocina sin requerir reautenticaciones repetitivas.

4.2 Mecanismo de Renovación (Refresh Tokens)

Para equilibrar seguridad y experiencia de usuario en sistemas de alta exigencia:

- **Access Token:** Corta duración (ej. 15 minutos), utilizado en cada petición a la API.
- **Refresh Token:** Larga duración (ej. 7 a 30 días), almacenado de forma persistente y segura. Su única función es comunicarse con un endpoint dedicado (`POST /api/auth/refresh`) para emitir un nuevo Access Token cuando el anterior expira, sin obligar al usuario a reescribir su contraseña.

4.3 Comportamiento del Cliente ante la Expiración

Cuando un Access Token expira, el backend rechaza la petición con código HTTP `401 Unauthorized`. El cliente web (React) debe contar con un interceptor que capture dicho código: si existe un Refresh Token, solicita uno nuevo en segundo plano y reintenta la petición; si no existe o el token es irrecuperable, limpia el almacenamiento y redirige automáticamente al usuario a la vista de `/login`.

5. Comparativa de Almacenamiento en React: Riesgos de Seguridad

La selección del medio de almacenamiento del JWT en el navegador determina la superficie de ataque frente a dos vulnerabilidades principales: **XSS (Cross-Site Scripting)** y **CSRF (Cross-Site Request Forgery)**.

Mecanismo de Almacenamiento	Persistencia	Exposición a XSS	Exposición a CSRF	Facilidad de Envío en API
`localStorage`	Persiste al cerrar pestaña y navegador.	**Alta:** Accesible por cualquier script JavaScript que corra en el origen.	**Nula:** No se envía automáticamente por el navegador.	Directa (lectura en código y armado manual del header `Bearer`).

Mecanismo de Almacenamiento	Persistencia	Exposición a XSS	Exposición a CSRF	Facilidad de Envío en API
`sessionStorage`	Se destruye al cerrar la pestaña actual.	**Alta:** Accesible por cualquier script JavaScript inyectado.	**Nula:** Requiere lectura e inserción programática.	Directa (lectura en código e inserción en header).
`Cookie httpOnly`	Configurable (sesión o tiempo fijo con `Max-Age`).	**Inmune:** El navegador bloquea el acceso de JavaScript a la cookie (`document.cookie`).	**Alta:** El navegador adjunta la cookie automáticamente en cualquier petición cruzada.	Automática (requiere flags `SameSite` y tokens Anti-CSRF).

5.1 Análisis de Vectores de Ataque

- **Riesgo en `localStorage` / `sessionStorage` (XSS):** Si la aplicación frontend es vulnerable a inyecciones de código malicioso (a través de dependencias desactualizadas o falta de sanitización de inputs), un atacante puede ejecutar `localStorage.getItem('token')` y exfiltrar las credenciales a un servidor externo.
- **Riesgo en `Cookies httpOnly` (CSRF):** Si bien elimina el robo directo por JavaScript, permite que un sitio web malicioso induzca al navegador de la víctima a ejecutar peticiones no autorizadas contra la API mientras la sesión esté activa, a menos que se configure estrictamente `SameSite: Lax` o `SameSite: Strict` y se implemente validación de cabeceras de origen.

6. Buenas Prácticas de Seguridad en la Implementación

- **Gestión de la Clave Secreta (`JWT_SECRET`):** Debe ser una cadena criptográficamente fuerte (mínimo 256 bits generados aleatoriamente), alojarse estrictamente en variables de entorno locales (`.env`), registrar dicho archivo en `.gitignore` y versionar únicamente un archivo `.env.example` con la estructura requerida.
- **Minimización de Datos en el Payload:** Incluir exclusivamente los identificadores estrictamente necesarios para autorizar la petición (por ejemplo: `id` del usuario). Queda estrictamente prohibido almacenar hashes de contraseñas, información confidencial del negocio o datos sensibles.
- **Cifrado de Transporte (HTTPS):** En entornos de producción, todas las comunicaciones entre cliente y servidor deben cursar sobre TLS/HTTPS para prevenir ataques de interceptación en tránsito (Man-in-the-Middle) que capturen los encabezados de autorización.

7. Decisión Técnica Adoptada para DarkKitchen Hub y Justificación

Para el desarrollo del proyecto **DarkKitchen Hub**, se toma la decisión arquitectónica de almacenar el JWT en **`localStorage`** y gestionarlo mediante un contexto de autenticación en React (`AuthContext`), complementado con inyección manual en los servicios de API.

Justificación Técnica:

- **Alineación con los Requerimientos del Proyecto:** La especificación técnica exige que cada llamada del módulo `services/` añada explícitamente el encabezado `Authorization: Bearer <token>` mediante interceptores o wrappers de fetch, lo cual requiere que el cliente tenga acceso programático al valor del token.
- **Desacoplamiento de Dominios y Puertos:** En el entorno de desarrollo y despliegue modular, el frontend (React en puerto 5173 / Vite) y el backend (Express en puerto 3000) operan en orígenes distintos. El uso de `localStorage` con cabeceras `Authorization` explícitas evita complicaciones transfronterizas de CORS asociadas a cookies de terceros (`credentials: 'include'`).
- **Mitigación del Riesgo XSS:** Dado que la aplicación no permite la carga de HTML dinámico no saneado de terceros y utiliza React (que escapa automáticamente los valores renderizados en el DOM virtual), el vector de ataque por XSS se encuentra razonablemente mitigado a nivel de arquitectura.

8. Fuentes Consultadas y Referencias Técnicas

- **Internet Engineering Task Force (IETF):** RFC 7519 - JSON Web Token (JWT) Specification.
- **OWASP Foundation:** JSON Web Token Cheat Sheet for Java/Node Developers.
- **OWASP Foundation:** Cross-Site Scripting (XSS) and HTML5 Local Storage Security.
- **Auth0 / Okta:** JSON Web Token Introduction and Architecture Guide.
- **Documentación Oficial de `jsonwebtoken` (npm):** Librería jsonwebtoken para Node.js.